COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	07/23/2026	**DATE SUBMITTED:**	07/272026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 3	**Linked Lists**

1. PROCEDURE OUTPUT

Screenshot	Program:
	`#include <iostream>`
	`#include "simply_LL.h"`
	
	`int main() {`
	
	`/// creating an empty list`
	`SimplyList<char>* head = new SimplyList<char>;`
	`SimplyList<char>* second = new SimplyList<char>;`
	`SimplyList<char>* third = new SimplyList<char>;`
	`SimplyList<char>* fourth = new SimplyList<char>;`
	`SimplyList<char>* fifth = nullptr;`
	
	`// assign the data`
	`head->data = 'T';`
	`head->next = second;`
	
	`second->data = 'R';`
	`second->next = third;`
	
	`third->data = 'O';`
	`third->next = fourth;`
	
	`fourth->data = 'Y';`
	`fourth->next = nullptr;`

Discussion	This code outputs every single character in the console. Every character is assigned to new memory; each parameter needs to be pointed to the data using the arrow pointer to assign it to a character and point it to the next parameter, unless that is the last parameter; then it should point to nullptr. This will output T->R->O->Y, since there is an arrow output in the simply_LL.h file. To improve this, the simply_LL.h file should create a createNode function so that it would be assigned while creating the list directly.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot

Traversal	Program:
	`std::cout << "Original List: " <<std::endl;`
	`listTraversal(head);`
	
	Output:
	`Original List:`
	`TROY`

Insertion at head	Program:
	`std::cout <<"\nInserting a new node at the head: " <<std::endl;`
	`sllinsertHead('Q', &head);`
	`listTraversal(head);`
	
	Output:
	`Inserting a new node at the head:`
	`QTROY`

Insertion at any part of the list	Program:
	`std::cout <<"\nInserting a new node at the third: "<<std::endl;`
	`sllGeneralInsert('X', head->next->next);`
	`listTraversal(head);`
	
	Output:
	`Inserting a new node at the third:`
	`QTRXOY`

Insertion at the end	Program:
	`std::cout <<"\nInserting a new node at the END: "<<std::endl;`
	`sllinsertEnd('U', &head);`
	`listTraversal(head);`
	
	Output:
	`Inserting a new node at the END:`
	`QTRXOYU`

Deletion of a node	Program:
	`std::cout <<"\ndeleting a node: " <<std::endl;`
	`sllDeleteNode('Q', &head);`
	`listTraversal(head);`
	
	Output:

	deleting a node: TRXOYU

Table 3-2. Code for the List Operations

a.	**Source Code**	6 /// creating an empty list 7 SimplyList<char>* head = new SimplyList<char>; 8 SimplyList<char>* second = new SimplyList<char>; 9 SimplyList<char>* third = new SimplyList<char>; 10 SimplyList<char>* fourth = new SimplyList<char>; 11 SimplyList<char>* fifth = new SimplyList<char>; 12 SimplyList<char>* sixth = new SimplyList<char>; 13 14 // assign the data 15 head->data = 'C'; 16 head->next = second; 17 18 second->data = 'P'; 19 second->next = third; 20 21 third->data = 'E'; 22 third->next = fourth; 23 24 fourth->data = '1'; 25 fourth->next = fifth; 26 27 fifth->data = '0'; 28 fifth->next = sixth; 29 30 sixth->data = '1'; 31 sixth->next = nullptr; 32 33 std::cout << "Original List: " <<std::endl; 34 listTraversal(head);
	Console	Original List: CPE101

b.	**Source Code**	std::cout <<"\nInserting a new node at the head: " <<std::endl;
		sllinsertHead('G', &head);
		listTraversal(head);

	Console	Inserting a new node at the head: GCPE101

c.	**Source Code**	std::cout <<"\nInserting a new node at the foruth: "<<std::endl;
		sllGeneralInsert('E', head->next->next->next);
		listTraversal(head);

	Console	Inserting a new node at the foruth: GCPEE101
d.	**Source Code**	std::cout <<"\nDeleting a node: " <<std::endl; sllDeleteNode('P', &head);
	Console	Deleting a node: GEE101

e.	**Source Code**	std::cout <<"\nDeleting a node: " <<std::endl;
		sllDeleteNode('P', &head);
		sllDeleteNode('C', &head);

	Console	Deleting a node: GEE101

f.	**Source Code**	std::cout <<"\nDeleting a node: " <<std::endl;
		sllDeleteNode('P', &head);
		sllDeleteNode('C', &head);
		listTraversal(head);

	Console	Deleting a node: GEE101

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis

Traverse:
`#include <iostream>`
`#include "simply_LL.h"`
`#include "double_LL.h"`
 	
`int main() {`
`  // creating the first code 'C' 'P' 'E'`
`  DoubleList<char>* head = createNode('C');`
`  DoubleList<char>* second = createNode('P');`
`  DoubleList<char>* third = createNode('E');`
 	
`  // Link the nodes together`
`  head->next = second;`
`  second->prev = head;`
 	
`  second->next = third;`
`  third->prev = second;`
 	
`  dllTraverse(head);`
 	
Output:
`Forward:`
`C P E`
 	
`Backward:`
`E P C`
	This code traverses each character and outputs to the console. The double_LL.h file has a createNode function that will directly assign each parameter to a character instead of making a long, tedious code. The nodes are also linked to next and prev to satisfy and double linked list. Then it will traverse and output “C P E” both forwards and backwards.

Insertion at Head:
`std::cout <<"Insert Head Testing: " <<std::endl;`
`dllInsertHead('X', &head);`
`dllTraverse(head);`
 	
Output:
`Insert Head Testing:`
`Forward:`
`X C P E`
 	
`Backward:`
`E P C X`
	This code inserts the letter given to the head. The dllInsertHead function allows the character ‘X’ to be inserted at the head of the function and outputs “X C P E” both forwards and backwards.

Insertion at End:
`std::cout <<"Insert End Testing: " <<std::endl;`
`dllInsertEnd('Q', head);`
`dllTraverse(head);`
 	
Output
	This code inserts the letter given to the end of the list. The dllInsertEnd function makes the character ‘Q’ insert itself to the end of the node, and once it’s output by the dllTraverse function, it will output X C P E Q both forwards and backwards.

Insert End Testing: Forward: X C P E Q Backward: Q E P C X	

Insertion at any Node:
`std::cout <<"\nGeneral Insert Testing: " <<std::endl;`
`dllGeneralInsert('Y', head->next);`
`dllTraverse(head);`
 	
Output:
General Insert Testing:
Forward:
X C Y P E Q
 	
Backward:
Q E P Y C X
	This code inserts the letter given into the middle of the list. The dllGeneralInsert function allows the character 'Y' to be inserted in the middle of the list, more specifically at the 2nd node, and outputs using the traverse function: X C Y P E Q, both forwards and backwards.

Deletion of a Node:
 	
`std::cout <<"\nDeleting a node: " <<std::endl;`
`dllDeleteNode('C', &head);`
`dllTraverse(head);`
 	
Output:
Deleting a node:
Forward:
X Y P E Q
 	
Backward:
Q E P Y X
	This code will find the character given and delete it if it finds that character. The dllDeleteNode function works by giving it a character and will find it in the list. If it finds the character, it will delete of the list. In this program, it asks the program to delete 'C' in the list, and outputs X Y P E Q both forwards and backwards.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once)

Main Program:

```cpp
// import song_playylist.h
#include <iostream>
#include <string>
#include "song_playlist.h"

int main() {
    // declare parameters for main menu and switch cases
    int choice;
    std::string Exit;
    std::string song;
    Song_Playlist* head = nullptr;

 // while loop that will loop until choice == 5
while (choice != 5) {
    // main menu
    std::cout <<"\n-----Main Menu-----" <<std::endl;
    std::cout <<"1. Add a song" <<std::endl;
    std::cout <<"2. Delete a song" <<std::endl;
    std::cout <<"3. Check Playlist" <<std::endl;
    std::cout <<"4. PLay all songs" <<std::endl;
    std::cout <<"5. Exit" <<std::endl;
    std::cout <<"\nEnter your choice: ";
    std::cin >> choice;

// switch case for choice
switch (choice) {
    // Adds song
    case 1: {
        std::cout <<"Please enter the song you want to add: ";
        std::cin.ignore();
        std::getline(std::cin, song);
        SPAddSong(song, &head);
        std::cout <<"Song Added!" <<std::endl;
        break;
    }

    // Deletes song
    case 2: {
        std::cout <<"Please enter the song you want to delete: ";
        std::cin >> song;
        SPDeleteSong(song, &head);
        break;
    }

//  Checks the contents of the playlist
case 3: {
    std::cout <<"List of Songs: " <<std::endl;
    SPTraverse(head);
    std::cout <<"\nEnter any key to go back to the main menu" <<std::endl;
    std::cin >> Exit;
    break;
}

// Play the songs (Display all the songs once)
case 4: {
    std::cout <<"Now Playing:" <<std::endl;
    SPTraverse(head);
    std::cout <<"\nEnter any key to stop the playlist and go back to the main menu" <<std::endl;
    std::cin >> Exit;
    break;
}

//Exit case
case 5: {
    break;
}
}
}
}
```

Output:

Create a playlist using multiple songs.

```
-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 3
List of Songs:
There are no songs added yet.

Enter any key to go back to the main menu
1

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 1
Please enter the song you want to add: Never Gonna Give You Up
Song Added!

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 1
Please enter the song you want to add: We are Charlie Kirk
Song Added!

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 1
Please enter the song you want to add: Despacito
Song Added!
```

```
-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 1
Please enter the song you want to add: The End
Song Added!

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 3
List of Songs:
Never Gonna Give You Up
We are Charlie Kirk
Despacito
The End

Enter any key to go back to the main menu
d
```

Remove a song from the playlist.

```
-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 2
Please enter the song you want to delete: Despacito
Song Deleted!

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 3
List of Songs:
Never Gonna Give You Up
We are Charlie Kirk
The End

Enter any key to go back to the main menu
q
```

Add songs to the playlist.

```
-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 1
Please enter the song you want to add: Multo
Song Added!

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 3
List of Songs:
Never Gonna Give You Up
We are Charlie Kirk
The End
Multo

Enter any key to go back to the main menu
t
```

Play songs in a loop

```
-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 4
Now Playing:
Never Gonna Give You Up
We are Charlie Kirk
The End
Multo

Enter any key to stop the playlist and go back to the main menu
o

-----Main Menu-----
1. Add a song
2. Delete a song
3. Check Playlist
4. PLay all songs
5. Exit

Enter your choice: 5
PS C:\Users\ASUS TUF\Documents\Activities_C++\HOA3p1>
```

Conclusion:

In conclusion, linked lists are data structures that link elements (called nodes) using pointers. Linked lists can be helpful because arrays are limited and can make you do lengthy and tedious code when trying to change or insert a new element. It works by creating a list with two portions in each node: the content and the pointer, and each of them goes together to make the linked list functional.

In the procedure, it is asked to create a singly and double linked list to traverse, insert at head, any node, and end, and delete a node. We are helped with this code, so most of the work was already done and just needs to be modified to follow the procedure. We are also shown how to create a more efficient way to insert a character in the list in a double-linked list.

In the supplementary activity, the linked list is a bit more complicated since instead of a character, we use a string to insert songs in the playlist. Since there are spaces in some songs, we used cin.ignore() and std::getline to read strings with spaces as one string instead of being separated and causing problems. Most of the code for the circular linked lists is similar to the singly linked list, so I just need to copy the code and make modifications.

Overall, making the program was a bit of a struggle since it was filled with many unnecessary lines that made the program buggy and did not satisfy the supplementary activity, but with debugging help from AI, I managed to get the code working and make it fully functional. I would want to improve a circular linked list because I mainly struggled with implementing it for the supplementary activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT to help me with debugging and fixing the code without giving the work to the AI**
I used Grammarly to at least fix most of my grammar mistakes to polish the conclusion and the analysis

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY